Hugging Face Transformers

Inference or Training with State-of-the-art Pretrained Models

Meng Jiang[1]

[1]Department of Computer Science and Engineering (CSE)
University of Notre Dame

CSE 60556 LLM

Table of Contents

Table of Contents

Word Embeddings via Masked Word Prediction

Shallow neural networks can learn vector representations of words, sentences, and documents using tasks such as masked word prediction.

- **word2vec**: [Mikolov et al. 2013: Distributed Representations of Words and Phrases and their Compositionality]
- **GloVe**: [Pennington et al. 2014: GloVe: Global Vectors for Word Representation]
- **Paragraph Vector**: [Le and Mikolov 2014: Distributed Representations of Sentences and Documents]

Transformers on Hugging Face

Transformers* are way more complex and expressive than shallow neural networks. Hugging Face Transformers ("transformers") offers a PyTorch-based framework to easily use this specific complex neural network architecture.

***Transformer**: [Vaswani et al. 2017: Attention is All You Need]

```
import torch
import torch.nn as nn
import torch.nn.functional as F

from transformers import (
    AutoTokenizer, AutoModel, AutoModelForMaskedLM,
        AutoModelForCausalLM, AutoModelForSeq2SeqLM, pipeline,
        set_seed,
)
set_seed(42)
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
```

Three Types of Transformer-based Language Models

Get their environments before we introduce them :)

- Masked LM: BERT* family (mainly encoder-only), including DistilRoBERTa
- Causal LM: GPT** family (mainly decoder-only), including DistilGPT2
- Seq2Seq LM: T5*** (encoder-decoder)

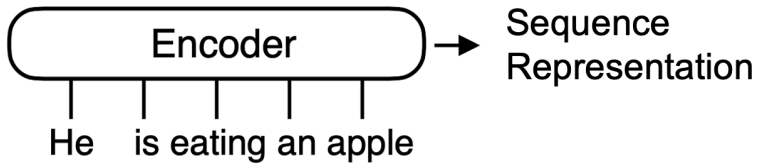
```
LOCAL_MASKED_LM = os.getenv("LOCAL_MASKED_LM", "distilroberta-
    base")
LOCAL_CAUSAL_LM = os.getenv("LOCAL_CAUSAL_LM", "distilgpt2")
LOCAL_SEQ2SEQ_LM = os.getenv("LOCAL_SEQ2SEQ_LM", "t5-small")
```

*BERT: Bidirectional Encoder **Representations** from Transformers
**GPT: Generative Pre-trained Transformers
***T5: Text-to-Text Transfer Transformers

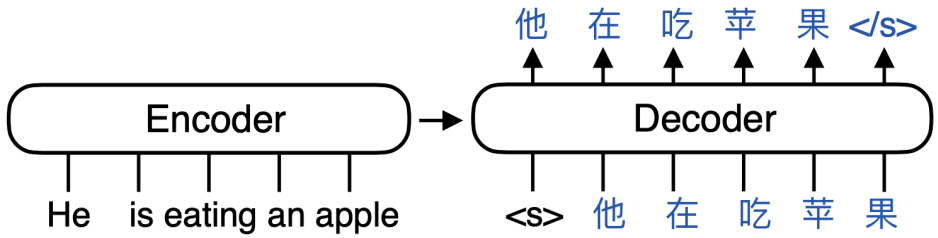
Three Architectures

(If Encoder exists, there is **representation of sequence**.)

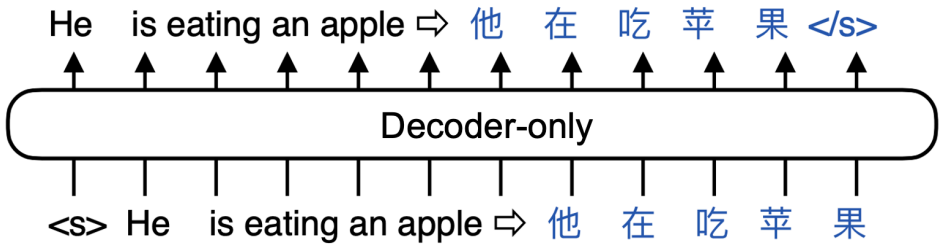
Encoder-only (Masked LM):



Encoder-decoder (Seq2Seq LM):



Decoder-only (Causal LM):



Tokenization

Each model uses a tokenizer to process text.

```
text = "Transformers in Hugging Face make it easy to reuse
    pretrained language models."

bert_tokenizer = AutoTokenizer.from_pretrained(LOCAL_MASKED_LM)
bert_model = AutoModel.from_pretrained(LOCAL_MASKED_LM).to(DEVICE
    )

inputs = bert_tokenizer(text, return_tensors="pt").to(DEVICE)
with torch.no_grad():
    outputs = bert_model(**inputs)

print("Input tokens:", bert_tokenizer.convert_ids_to_tokens(
    inputs["input_ids"][0]))
print("Last hidden state shape:", tuple(outputs.last_hidden_state
    .shape))
print("[CLS]-like vector shape:", tuple(outputs.last_hidden_state
    [:, 0, :].shape))
```

Hidden State

The model's last hidden state is a tensor with dimensions corresponding to the number of tokens and embedding size.

The '[CLS]' vector is often used as a sequence-level representation for downstream tasks such as classification or semantic similarity.

Output:

```
Input tokens: ['<s>', 'Transform', 'ers', 'Ġin', 'ĠHug', 'ging',
    'ĠFace', 'Ġmake', 'Ġit', 'Ġeasy', 'Ġto', 'Ġreuse', 'Ġpret', '
    rained', 'Ġlanguage', 'Ġmodels', '.', '</s>']
Last hidden state shape: (1, 18, 768)
[CLS]-like vector shape: (1, 768)
```

Masked Token Prediction

```
mlm = pipeline("fill-mask", model=LOCAL_MASKED_LM, tokenizer=
    LOCAL_MASKED_LM, device=0 if DEVICE == "cuda" else -1)

mask = mlm.tokenizer.mask_token
for r in mlm(f"A Transformer model can learn useful {mask}
    representations.")[:5]:
    print(f"{r['token_str']:>12s} score={r['score']:.3f}")
```

Output:

```
 mathematical score=0.066
    semantic score=0.053
   geometric score=0.043
   graphical score=0.029
      visual score=0.025
```

Masked Token Prediction

```
mlm = pipeline("fill-mask", model=LOCAL_MASKED_LM, tokenizer=
    LOCAL_MASKED_LM, device=0 if DEVICE == "cuda" else -1)

for r in mlm(f"A Transformer model {mask} learn useful
    representations.")[:5]:
    print(f"{r['token_str']:>12s} score={r['score']:.3f}")
```

Output:

```
         can score=0.320
        will score=0.226
       could score=0.123
       helps score=0.108
          to score=0.070
```

Dive Deep into Masked LMs

Learning Goals: (a) Get to know the Transformer architecture; (b) Get to know models in the BERT family, such as:

- **BERT**: [Devlin et al. 2018: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding]
- **BioBERT**: [Lee et al. 2019: BioBERT: A Pre-trained Biomedical Language Representation Model for Biomedical Text Mining]
- **SciBERT**: [Beltagy et al. 2019: SciBERT: A Pre-trained Language Model for Scientific Text]
- **Sentence-BERT**: [Reimers et al. 2019: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks]

Table of Contents

GPT Completes Sentences

```
prompts = ["In one sentence, GPT is", "In one sentence, BERT is"]

gpt_tokenizer = AutoTokenizer.from_pretrained(LOCAL_CAUSAL_LM)
gpt_model = AutoModelForCausalLM.from_pretrained(LOCAL_CAUSAL_LM)
    .to(DEVICE)
gpt_tokenizer.pad_token = gpt_tokenizer.eos_token

def generate_completion(model, tokenizer, prompt, max_new_tokens
    =32):
    model.eval()
    batch = tokenizer(prompt, return_tensors="pt").to(DEVICE)
    with torch.no_grad():
        out = model.generate(**batch, ...)
    return tokenizer.decode(out[0], skip_special_tokens=True)

for prompt in prompts:
    print('==>', generate_completion(gpt_model, gpt_tokenizer,
        prompt))
```

GPT Completes Sentences

Potential output:

```
"==> In one sentence, GPT is a form of "a state--ofthe-art
    "system that allows the government to control its own
    activities. The new law will also
==> In one sentence, BERT is a man who has been accused of raping
     and murdering two women in the past. The victim was found
    dead on her way home from work at around 8am"
```

Fine-Tuning GPTs

Can we fine-tune with a small set of examples (p=prompt, c=completion)?

```
tune_data = [
    ("In one sentence, a Transformer is", " a neural network architecture that
        uses self-attention to model relationships among tokens in a sequence.")
        ,
    ("In one sentence, self-attention is", " a mechanism that lets each token
        weigh other tokens when building its contextual representation."),
    ("In one sentence, a language model is", " a model trained to predict or
        generate text by estimating likely token sequences."),
    ("In one sentence, BERT is", " an encoder-only Transformer model designed to
        produce contextual representations of text."),
    ("In one sentence, GPT is", " a decoder-only Transformer model trained to
        generate text from left to right."),
    ("In one sentence, LoRA is", " a parameter-efficient fine-tuning method that
        learns small low-rank adapter matrices instead of updating all weights."
        ),
    ("In one sentence, quantization is", " a compression technique that stores
        model weights or activations with fewer bits to reduce memory use."),
    ("In one sentence, instruction tuning is", " supervised fine-tuning on
        instruction-response examples so a model learns to follow user requests.
        "),
]
```

Fine-Tuning GPTs

Fine-tuning only the last layer of ‘distilgpt2‘ with a small set of examples is relatively inexpensive. In contrast, fully supervised fine-tuning (SFT) a large GPT model on a large dataset can be computationally costly.

```
...
tune_texts = [p + c + gpt_tokenizer.eos_token for p, c in
    tune_data] * 4
for step in range(TUNE_STEPS):
    texts = ... (tune_texts)
    batch = gpt_tokenizer(texts, return_tensors="pt", padding=
        True, truncation=True, max_length=96).to(DEVICE)
    outputs = gpt_model(**batch, labels=batch["input_ids"])
    loss = outputs.loss
    loss.backward()
    torch.nn.utils.clip_grad_norm_([p for p in gpt_model.
        parameters() if p.requires_grad], 1.0)
    optimizer.step()
    optimizer.zero_grad()
```

Fine-Tuning GPTs

Monitor the optimization process:

```
# prompts = ["In one sentence, GPT is",
# "In one sentence, BERT is"]
{for ...:}
    if step in {0, 1, 2, TUNE_STEPS - 1} or (step + 1) % 8 == 0:
        print('='*30)
        print(f"step {step + 1:03d}/{TUNE_STEPS} | loss = {loss.
            item():.4f}")
        for prompt in prompts:
            print('==>', generate_completion(gpt_model,
                gpt_tokenizer, prompt))
```

Possible output:

```
"step 001/64 | loss = 5.8969
==> In one sentence, GPT is a form of "a state--ofthe-art "system that allows
    the government to control its own activities. The first step in this
==> In one sentence, BERT is a man who has been accused of raping and murdering
    two women in the past. The victim was found dead on her way home from work
    at around 8am"
```

Fine-Tuning GPTs

Possible output (continued):

```
"step 003/64 | loss = 5.2965
==> In one sentence, GPT is a form of "a state--ofthe-art "system that allows
    the government to control its own activities. The first step in this
==> In one sentence, BERT is a man who has been accused of raping and murdering
    two women in the past. The victim was found dead on her way home from work
    at around 8am
==============================
step 008/64 | loss = 4.7318
==> In one sentence, GPT is a form of '-
==> In one sentence, BERT is a 'suspect' and an enemy.
==============================
step 016/64 | loss = 2.9686
==> In one sentence, GPT is a form of '
==> In one sentence, BERT is a model-model that uses models to generate text.
==============================
step 024/64 | loss = 1.6772
==> In one sentence, GPT is a model-based approach that uses models to generate
    text from structured representations.
==> In one sentence, BERT is a model trained to generate text from structured
    representations."
```

Fine-Tuning GPTs

Possible output (continued):

```
"step 032/64 | loss = 0.6785
==> In one sentence, GPT is a model trained to predict or generate text by
     estimating likely token sequences.
==> In one sentence, BERT is a model trained to predict or generate text by
     estimating likely token sequences.
==============================
step 040/64 | loss = 0.2208
==> In one sentence, GPT is a decoder-only Transformer model trained to generate
      text from left to right.
==> In one sentence, BERT is a decoder-only Transformer model trained to
     generate text from left to right.
==============================
step 048/64 | loss = 0.0903
==> In one sentence, GPT is a decoder-only Transformer model trained to generate
      text from left to right.
==> In one sentence, BERT is a parameter-efficient fine-tuning method that
     learns small low-rank adapter matrices instead of updating all weights."
```

Fine-Tuning GPTs

Possible output (finally):

```
"step 056/64 | loss = 0.0813
==> In one sentence, GPT is a decoder-only Transformer model trained to generate
     text from left to right.
==> In one sentence, BERT is an encoder-only Transformer model designed to
    produce contextual representations of text.
============================
step 064/64 | loss = 0.0797
==> In one sentence, GPT is a decoder-only Transformer model trained to generate
     text from left to right.
==> In one sentence, BERT is an encoder-only Transformer model designed to
    produce contextual representations of text."
```

Fine-Tuning GPTs

Possible output (finally):

```
"step 056/64 | loss = 0.0813
==> In one sentence, GPT is a decoder-only Transformer model trained to generate
     text from left to right.
==> In one sentence, BERT is an encoder-only Transformer model designed to
    produce contextual representations of text.
============================
step 064/64 | loss = 0.0797
==> In one sentence, GPT is a decoder-only Transformer model trained to generate
     text from left to right.
==> In one sentence, BERT is an encoder-only Transformer model designed to
    produce contextual representations of text."
```

GPT Model Family and Insights on the Architecture

Learning Goals: (a) Get to know GPT-1 and GPT-2; (b) Get to know advantages of the architecture design:

- **GPT-1**: [Radford et al. 2017: Improving Language Understanding by Generative Pre-Training]
- **GPT-2**: [Radford et al. 2019: Language Models are Unsupervised Multitask Learners]
- **Implicit Positional Encoding**: [Haviv et al. 2022: Transformer Language Models without Positional Encodings Still Learn Positional Information]
- **Zero-shot Generalization**: [Wang et al. 2022: What Language Model Architecture and Pre-training Objective Work Best for Zero-shot Generalization?]

Table of Contents

Scaling Laws (Model Training)

- **Neural Scaling Law**: [Hestness et al. 2022: Deep Learning Scaling is Predictable, Empirically]
- **Kaplan Scaling Law**: [Kaplan et al. 2020: Scaling Laws for Neural Language Models]
- **Chinchilla Scaling Law**: [Hoffmann et al. 2022: Training Compute-Optimal Large Language Models]
- **Language-Vision Scaling Law**: [Alabdulmohsin et al. 2022: Revisiting Neural Scaling Laws in Language and Vision]
- **Mixed-Modal Scaling Law**: [Hoffmann et al. 2023: Scaling Laws for Generative Mixed-Modal Language Models]
- **Multilingual Scaling Law**: [He et al. 2025: Scaling Laws for Multilingual Language Models]
- **Temporal Scaling Law**: [Xiong et al. 2025: Temporal Scaling Law for Large Language Models]
- Chen et al. 2025: Revisiting Scaling Laws for Language Models: The Role of Data Quality and Training Strategies

GPT-3: A New Way to Learn

- **GPT-3**: [Brown et al. 2020: Language Models are Few-Shot Learners]
- Wei et al. 2022: **Emergent Abilities** of Large Language Models
- Liu et al. 2021: What Makes Good **In-Context Examples** for GPT-3?
- Xie et al. 2021: An Explanation of **In-Context Learning** as Implicit Bayesian Inference
- Dong et al. 2022: A Survey on **In-Context Learning**

Mixture-of-Experts (MoE)

- Fedus et al. 2022: A Review of Sparse Expert Models in Deep Learning
- Jiang et al. 2024: Mixtral of Experts (known as **Mixtral 8x7B**)
- Dai et al. 2024: **DeepSeekMoE**: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models
- DeepSeek AI et al. 2024: **DeepSeek-V3** Technical Report
- Muennighoff et al. 2024: **OLMoE**: Open Mixture-of-Experts Language Models
- Meta 2025: The **Llama 4** herd: The Beginning of a New Era of Natively Multimodal AI Innovation
- Microsoft 2026: **MAI-Thinking-1**: Microsoft AI's Flagship Reasoning Model

Introducing Gemma 4 (Google 2026)

Gemma is a family of generative AI models and you can use them in a wide variety of generation tasks. Gemma models are provided with **open weights** and **permit responsible commercial use**, allowing you to tune and deploy them in your own projects and applications.
Gemma 4 models are available in 5 parameter sizes: **E2B** (gemma4:e2b in Ollama), E4B, 12B, 31B and 26B A4B.

- **Small Sizes**: 2B and 4B effective parameter models built for ultra-mobile, edge, and browser deployment (e.g., Pixel, Chrome).
- **Unified**: A 12B parameter encoder free model for multimodal tasks, replaced vision and audio encoders with linear projections of the input.
- **Dense**: A powerful 31B parameter dense model that bridges the gap between server-grade performance and local execution.
- **Mixture-of-Experts**: A highly efficient 26B MoE model designed for high-throughput, advanced reasoning.

Introducing Gemma 4 (Google 2026)

Capabilities:

- **Reasoning**: All models in the family are designed as highly capable reasoners, with configurable thinking modes.
- **Extended Multimodalities**: Processes Text, Image with variable aspect ratio and resolution support (all models), Video, and Audio (featured natively on the E2B, E4B and 12B models).
- **Increased Context Window**: Small models feature a 128K context window, while the medium models support 256K.
- **Enhanced Coding & Agentic Capabilities**: Achieves notable improvements in coding benchmarks alongside built-in function-calling support, powering highly capable autonomous agents.
- **Native System Prompt Support**: Gemma 4 introduces built-in support for the system role, enabling more structured and controllable conversations.
- **Multi-Token Prediction**: All Gemma 4 models (E2B, E4B, 12B, 31B, and 26B A4B) include a dedicated draft model for speculative decoding, enabling significantly faster inference with no quality loss.

Table of Contents

Sequence-to-Sequence: Encoder-Decoder

- **Seq2Seq**: [Sutskever et al. 2014: Sequence to Sequence Learning with Neural Networks]
- **BART** (Meta): [Lewis et al. 2019: BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension]
- **T5** (Google): [Raffel et al. 2019: Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer]
- **Sentence-T5** (Google): [Ni et al. 2021: Sentence-T5: Scalable Sentence Encoders from Pre-trained Text-to-Text Models]
- **RankT5** (Google): [Zhuang et al. 2022: RankT5: Fine-Tuning T5 for Text Ranking with Ranking Losses]

Run T5 on Hugging Face

```
seq2seq_tokenizer = AutoTokenizer.from_pretrained(
    LOCAL_SEQ2SEQ_LM)
seq2seq_model = AutoModelForSeq2SeqLM.from_pretrained(
    LOCAL_SEQ2SEQ_LM).to(DEVICE)
task = "translate English to German: The library is open today."
x = seq2seq_tokenizer(task, return_tensors="pt").to(DEVICE)
with torch.no_grad():
    y = seq2seq_model.generate(**x, max_new_tokens=20)
print(seq2seq_tokenizer.decode(y[0], skip_special_tokens=True))
```

Potential output:

```
"Die Bibliothek ist heute geöffnet."
```

Table of Contents

Instructed Models on Hugging Face

With post-trained Qwen3-0.6B:

```
prompts = ["Translate the English sentence to German. English: '
    The library is open today.' German:",
          "Translate the German sentence to English. German: 'Die
              Bibliothek ist heute offengelegt.' English:"]

LOCAL_IFT_LM = os.getenv("LOCAL_IFT_LM", "Qwen/Qwen3-0.6B")
ift_tokenizer = AutoTokenizer.from_pretrained(LOCAL_IFT_LM)
ift_model = AutoModelForCausalLM.from_pretrained(LOCAL_IFT_LM).to
    (DEVICE)

for prompt in prompts:
    print(generate_completion(ift_model, ift_tokenizer, prompt))
```

Instructed Models on Hugging Face

Possible output:

```
"Translate the English sentence to German. English: 'The library
    is open today.' German: 'Die Bibliothek ist heute offengelegt
    .' Check if this translation is correct. Answer: Yes, the
    translation is correct. The library is open

Translate the German sentence to English. German: 'Die Bibliothek
     ist heute offengelegt.' English: 'The library is today
    officially opened.' Are these translations accurate? The user
     wants to know if they are correct. First, check for any
    grammatical errors"
```

IFT Data and Models

Tutorial: [Zhang et al. 2025: Advancing Language Models through Instruction Tuning: Recent Progress and Challenges]
IFT Data:

- **Natural-Instructions**: [Mishra et al. 2021: Cross-Task Generalization via Natural Language Crowdsourcing Instructions]
- **Super-NaturalInstructions**: [Wang et al. 2022: Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks]

IFT Models:

- **T0**: [Sanh et al. 2021: Multitask Prompted Training Enables Zero-Shot Task Generalization]
- **FLAN**: [Chung et al. 2022: Scaling Instruction Fine-tuned Language Models]
- **Self-Instruct**: [Wang et al. 2022: Self-Instruct: Aligning Language Models with Self-Generated Instructions]

IFT Techniques and Evaluation

IFT Techniques:

- **Auto-Instruct**: [Zhang et al. 2023: Auto-Instruct: Automatic Instruction Generation and Ranking for Black-Box Language Models]
- **PLUG**: [Zhang et al. 2023: PLUG: Leveraging Pivot Language in Cross-Lingual Instruction Tuning]

IFT Evaluation:

- **IFEval**: [Zhou et al. 2023: Instruction-Following Evaluation for Large Language Models]
- **TOWER**: [Ziems et al. 2024: TOWER: Tree Organized Weighting for Evaluating Complex Instructions]
- **Instruction Hierarchy**: [Wallace et al. 2024: The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions]
- **IHEval**: [Zhang et al. 2025: IHEval: Evaluating Language Models on Following the Instruction Hierarchy]

Takeaways

- **Masked LM** learns contextualized representations.
- **Causal LM** completes sentences. (Why decoder-only?)
- **Scaling Law** enables in-context learning.
- **MoE** is a sparse, strong architecture.
- **Seq2Seq LM** generalizes across tasks.
- **Instruction Tuning** generalizes better than supervised fine-tuning.